

TP1_clean

January 23, 2025

```
[1]: import warnings
warnings.filterwarnings( "ignore", module = "findfont/.*" )
warnings.filterwarnings("ignore", category=UserWarning, module="matplotlib")

warnings.filterwarnings("ignore", category=UserWarning, module="matplotlib")
```

```
[2]: # use xkcd color palette
import matplotlib.pyplot as plt
import numpy as np
import matplotlib as mpl
#set xkcd
#plt.xkcd()
import matplotlib
```

1 2 Generation of synthetic Data

1.1 Step 1: Load the diffusion signal

```
[1]: import numpy as np
def get_X():
    X=np.loadtxt("x.txt")
    return X
X=get_X()
N=X.shape[0]
```

1.2 Step 2: Create T using exponential sampling strategy

```
[4]: ## Step 2: Create T using exponential sampling strategy
def construct_T(N=200):
    T_min=1
    T_max=1000
    T=np.zeros(N)
    for k in range(N):
        T[k]=T_min*np.exp(-(k)*np.log(T_min/T_max)/(N-1))
    return T
T=construct_T(N)
```

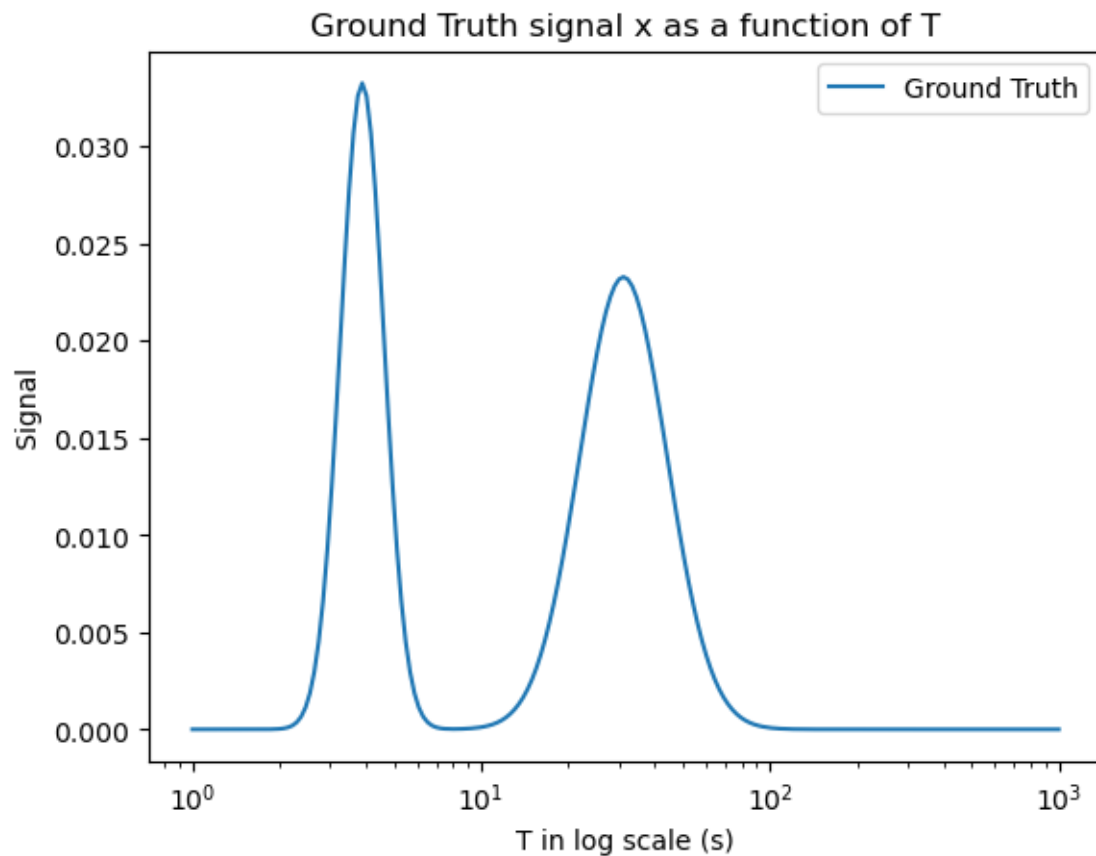
1.3 Step 3: Display the original signal x as a function of T

```
[5]: import matplotlib.pyplot as plt
def plot_signal(T,X,title,label):
    plt.plot(T,X,label=label)
    plt.xlabel("T in log scale (s)")
    plt.ylabel(" Signal ")
    plt.xscale("log")
    plt.title(title)
    plt.legend()

title="Ground Truth signal x as a function of T"
label="Ground Truth"
plot_signal(T,X,title,label=label)
# add log scale on horizontal axis

plt.xscale('log')

plt.show()
```



1.4 Step 4: Create t using regular sampling strategy

```
[6]: def construct_t():
    t_min=0
    t_max=1.5
    M=50
    t=np.zeros(M)
    for m in range(M):
        t[m]=t_min+(m)/(M-1)*(t_max-t_min)
    return t
t=construct_t()
t.shape
```

```
[6]: (50,)
```

1.5 Step 5: Construct matrix K

```
[ ]: def construct_K(T,t):
    M=len(t)
    N=len(T)
    K=np.zeros((M,N))
    for m in range(M):
        for n in range(N):
            K[m,n]=np.exp(-T[n]*t[m])
    return K
K=construct_K(T,t)
# compute the rank

print(np.linalg.matrix_rank(K))
np.linalg.matrix_rank(K.T @ K)
```

```
25
```

```
[ ]: 15
```

1.6 Step 6: Simulate the noisy data

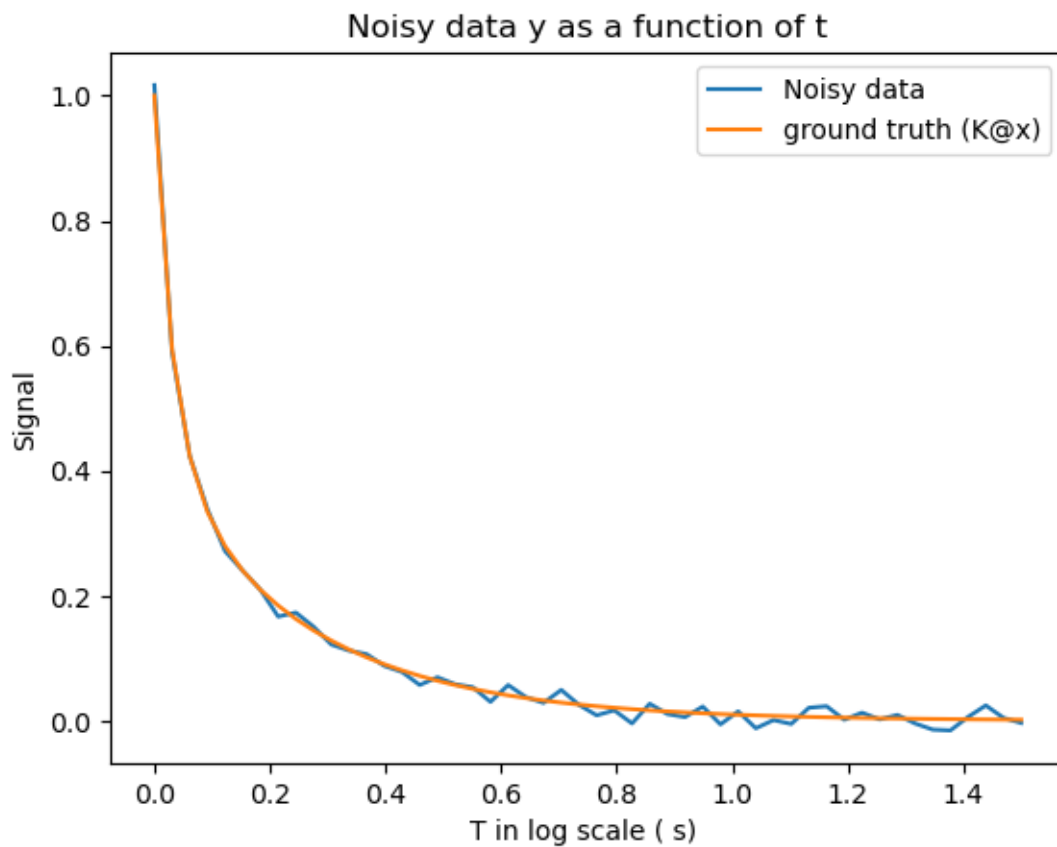
```
[8]: # Simulate the noisy data according to model (2), by taking  $w \sim N(0, 2IM)$ 
# with  $\sigma = 0.01 \sqrt{z(1)}$ , where  $z = Kx$ .
np.random.seed(7)

def get_y():
    mu=0
    z=np.dot(K,X)
    sigma=0.01*z[0]
    w=np.random.normal(mu,sigma,z.shape[0])
    # simulate it
```

```
y=np.dot(K,X)+w
return y
y=get_y()
```

1.7 Step 7 : Display the resulting noisy data y as a function of t

```
[9]: plt.plot(t,y,label="Noisy data")
plt.plot(t,K@X,label="ground truth (K@x)")
plt.xlabel("T in log scale ( s ) ")
plt.ylabel("Signal")
plt.title("Noisy data y as a function of t")
plt.legend()
plt.show()
```



2 Comparison of regularization strategies

```
[11]: # Template class for the several function to minimize
class fun_minimize:
    # generic class for the minimisation problem
    def __init__(self,*args,**kwargs):
        return
    def __call__(self, *args, **kws):
        # return f(x)
        return NotImplementedError
    def grad(self,
             *args, **kws):
        # return gradient of f(x)
        return NotImplementedError
    def proximal(self,*args,**kws):
        # return proximal operator of f(x)
        return NotImplementedError
```

$$\hat{x} = \arg \min_{x \in \mathbb{R}^N} \frac{1}{2} \|Kx - y\|^2 + \beta g(x)$$

On posera dans la suite $h(x)=f(x)+g(x)$

2.1 Smoothness prior

Smoothness prior:

$$(\forall x \in \mathbb{R}^N) \quad g(x) = \frac{1}{2} \|Dx\|^2$$

où $D \in \mathbb{R}^{N \times N}$ est l'opérateur du gradient discret, tel que:

$$(\forall n = \{1, \dots, N\}) \quad [Dx]^{(n)} = x^{(n)} - x^{(n-1)}$$

2.1.1 Step 1: Existence and uniqueness of solutions

f et g sont continues, convexes et différentiables, en tant que composées de fonctions différentiables. Montrons que $f + g$ est coercive.

On remarque d'abord que $\text{Ker}(D) = \text{Vect}(\mathbf{1})$. Donc $g(x) \xrightarrow{x \notin \text{Vect}(\mathbf{1}), |x| \rightarrow +\infty} +\infty$.

Soit $x \in \mathbb{R}^n$ qu'on décompose en $x = x_g + x_f$ avec x_g la projection de x sur $\text{Ker}(D)$ et x_f la projection sur un supplémentaire orthogonale de $\text{Ker}(D)$ (qui existe car on est en dimension finie).

- Si $x_f \neq 0$, alors $h(x) \geq \frac{1}{2} \|Dx\|^2 = \frac{1}{2} \|Dx_g\|^2$ qui tend vers plus infini quand $\|x_g\|^2$ tend vers plus infini car $D(x_g) \neq 0$.
- si $x_g \neq 0$, alors $h(x) \geq \frac{1}{2} \|K(x_f) - y\|^2$. Pour $x \in \text{Vect}(\mathbf{1})$, on remarque que $x \notin \text{ker}(K)$ donc $\frac{1}{2} \|K(x_f) - y\|^2$ tend vers plus infini quand $\|x_g\|^2$ tend vers plus infini.

Dans tous les cas, $f+g$ est coercive. Donc donc $h = f + g$ est coercive, semi-continue inférieurement et propre. **Elle admet donc un minimum**

Trouvons ledit minimum:

On peut calculer leurs différentielles et obtenir:

$$\nabla_x f(x) = K^T K x - 2K^T y$$

$$\nabla_x g(x) = D^T D x$$

On a que

$$\nabla_x h(x) = 0 \Leftrightarrow (K^T K + \beta D^T D)x = K^T y.$$

Montrons que l'inverse est ici bien défini.

$K^T K$ et $D^T D$ sont semi-définies positives. Alors, pour que $x \in \mathbb{R}^N \setminus \{\mathbf{0}\}$ appartienne à $\text{Ker}\{K^T K + \beta D^T D\}$, il faut qu'il appartienne à la fois à $\text{Ker}(D^T D)$ et $\text{Ker}(K^T K)$.

On sait que $\text{Ker}(D^T D) = \text{Vect}\{\mathbf{1}\}$. Puisque $K^T K$ est positive, $\text{Ker}(K^T K) \cap \text{Vect}\{\mathbf{1}\} = \{\mathbf{0}\}$ et $(K^T K + \beta D^T D)$ est donc inversible.

On a donc:

$$x = (K^T K + \beta D^T D)^{-1} K^T y$$

Cette forme close montre l'unicité du minimum.

2.1.2 Step 2: Propose approaches for solving Problem (4)

On peut donc calculer la forme close pour résoudre le problème. On vérifie au passage que la fonction est bien numériquement inversible.

2.1.3 Step 3: Implement solution and display restored signal

```
[12]: # Construct D
def construct_D(N):
    D=np.zeros((N,N))
    for n in range(N):
        D[n,n]=1
        D[n,(n-1)%N]==-1

    return D
D=construct_D(N)
```

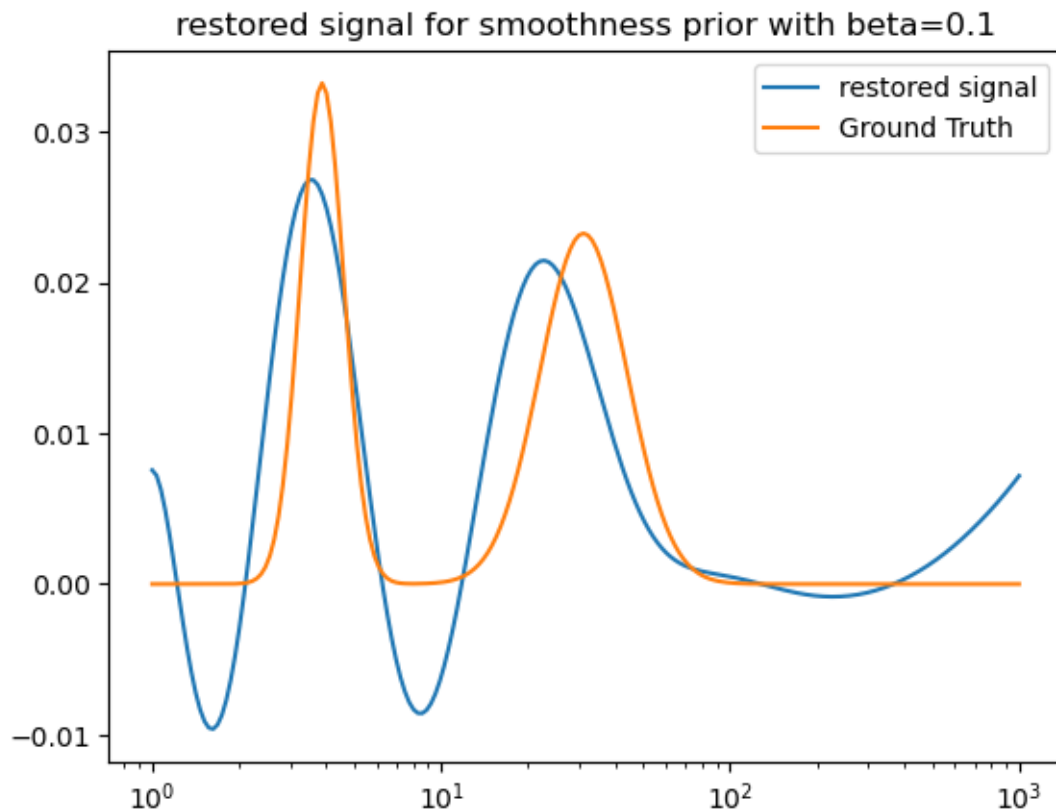
```
[13]: beta=0.1
```

```
[14]: # check that the matrix is invertible
A=K.T @K +beta*D.T @D
assert np.linalg.matrix_rank(A)==N
```

```
# Find the right x_chap
inv_A=np.linalg.inv(A)
x_chap= inv_A @ K.T @ y
```

2.1.4 Step 3: Implement solution and display restored signal

```
[15]: title="restored signal for smoothness prior with beta={}".format(beta)
label="restored signal"
plt.plot(T,x_chap,label="restored signal")
plt.plot(T,X,label="Ground Truth")
plt.legend()
plt.title(title)
plt.xscale('log')
```



2.1.5 Step 4: Compute normalized quadratic error

```
[18]: def normalize_quadratic_error(x_chap,x_barre):

    return np.linalg.norm(x_chap-x_barre)/np.linalg.norm(x_barre)
print("Normalized quadratic error",normalize_quadratic_error(x_chap,X))
```

Normalized quadratic error 0.562585783318674

2.2 Smoother prior+constraints

Smoothness prior + constraints:

$$(\forall x \in \mathbb{R}^N) \quad g(x) = \frac{1}{2} \|Dx\|^2 + \iota_{[x_{\min}:x_{\max}]^N}(x)$$

avec

$$(0 < x_{\min} < x_{\max})$$

les valeurs minimale et maximale du signal original $\bar{x}$.

Dans ce cas on peut poser

$$f(x) = \frac{\beta}{2} \|Dx\|^2 + \frac{1}{2} \|Kx - y\|^2$$
$$g(x) = \iota_{[x_{\min}:x_{\max}]^N}(x)$$

2.2.1 Step 1: Existence and uniqueness of solutions

On vérifie que

- f est continue
- g est l'indicatrice de l'intervalle $C = [x_{\min} : x_{\max}]^N$ qui est un intervalle convexe bornée .
Donc g est convexe

On peut donc réécrire $\arg \min_{x \in \mathbb{R}^n} h(x) = \arg \min_{x \in C} f(x)$. C est convexe et borné (en dimension finie) donc compact et f est continue, donc semi-continue inférieurement. Il existe alors un minimum sur l'intervalle. Comme f n'est pas a priori strictement convexe, le minimum n'est pas nécessairement unique

2.2.2 Step 2: Propose approaches for solving Problem (4)

On a que

- f est lisse et est ν lipschitzien avec ν la plus grande valeur propre de $(K^T K + \beta D^T D)$. En effet on a :

$$\forall (x, z) \in \mathbb{R}^N, \|\nabla(f)(x) - \nabla(f)(z)\| = \|(K^T K + \beta D^T D)(x - z)\| \leq \|K^T K + \beta D^T D\| \|x - z\|$$

- g est un ensemble fermé convexe non vide

On est dans les bonnes hypothèses de convergence de l'algorithme de gradient projeté, on va donc appliquer cette algorithme

2.2.3 Step 3: Implement solution and display restored signal

```
[19]: # calcule la projection de l'intervalle
def compute_Pc(x:"np.array",r_min:int,r_max:int):
    X_proj=np.zeros(x.shape)
    for i in range(x.shape[0]):
```



```

        xi=x[i]
        if xi>r_max:
            X_proj[i]=r_max
        if xi<r_min:
            X_proj[i]=r_min
        else:
            X_proj[i]=xi
    return X_proj
class g_smoothprior(fun_minimize):
    # It correspond to  $1/2||Kx-y||^2+1/2||Dx||^2$ 
    def __init__(self,K:"np.array",y:"np.array",D:"np.array",beta:float=1):
        self.K=K
        self.y=y
        self.D=D
        self.beta=beta
        return
    def __call__(self, x):
        return 1/2*(
            np.linalg.norm(self.K @ x-self.y)**2+self.beta* np.linalg.norm(self.
↪D @ x)**2)
    def grad(self,x:"np.array")->"np.array":
        return self.K.T @ self.K @ x - self.K.T @ y +self.beta* self.D.T @ self.
↪D @ x

x_test=np.zeros(N)

```

```

[20]: def projected_gradient(x0,lambda_n,num_iter,x_min,x_max,gamma,beta):
    x_n=x0
    y_n=x0
    g=g_smoothprior(K=K,y=y,D=D,beta=beta)
    loss=[]
    for n in range(num_iter):
        y_n=x_n-gamma*g.grad(x=x_n)

        x_n=x_n+lambda_n[n]*(compute_Pc(x=y_n,r_min=x_min,r_max=x_max)-x_n)
        if n%10000==0:
            print("iteration",n,"the loss is",g(x_n))
        loss.append(g(x_n))
    return x_n ,loss

```

```

[21]: # minimum values and maximum values of the original signal
x_min= X.min()
x_max= X.max()
# we gonna calculate v, the lipschitz gradient of g
g=g_smoothprior(K=K,y=y,D=D)

```

```
A= g.K.T @ g.K+ beta* g.D.T @ g.D
v=np.linalg.eigvals(A)
print("the largest eigenvalue is ",v.max())
v_max=v.max()
# so
```

the largest eigenvalue is 478.38203959403666

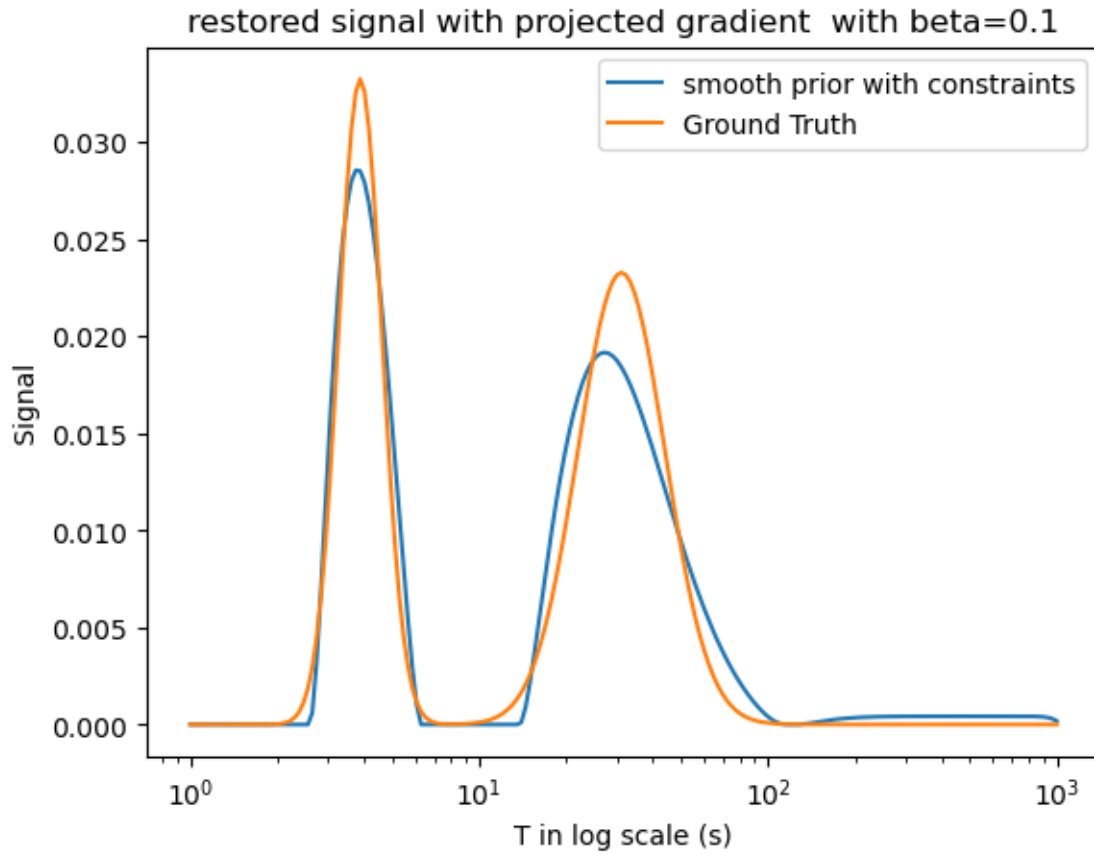
```
[22]: x0=np.zeros(N)
# x0 be a random vector
num_iter=50000
# Specify the beta parameter for g
beta=0.1
# gamma must be between 0 and 2/v_max
gamma=1/v_max
# lambda must be between 0 nad delta= 2-(gamma*v_max/2)
delta=2-(gamma*v_max/2)
lambda_n=[1/2*delta for k in range(num_iter)]

# Actual computation
x_chap_cst,loss=projected_gradient(x0,lambda_n,num_iter,x_min,x_max,gamma,beta=beta)
```

iteration 0 the loss is 0.15648777644970233
iteration 10000 the loss is 0.002828718694800753
iteration 20000 the loss is 0.002810190814708243
iteration 30000 the loss is 0.0028048044121181444
iteration 40000 the loss is 0.0028018715337175242

```
[23]: # display the restored signal
title="restored signal with projected gradient with beta={}".format(beta)
label="smooth prior with constraints"
plot_signal(T,x_chap_cst,title,label=label)
plt.plot(T,X,label="Ground Truth")
plt.xscale('log')
plt.legend()
```

```
[23]: <matplotlib.legend.Legend at 0x1dcee7e7440>
```



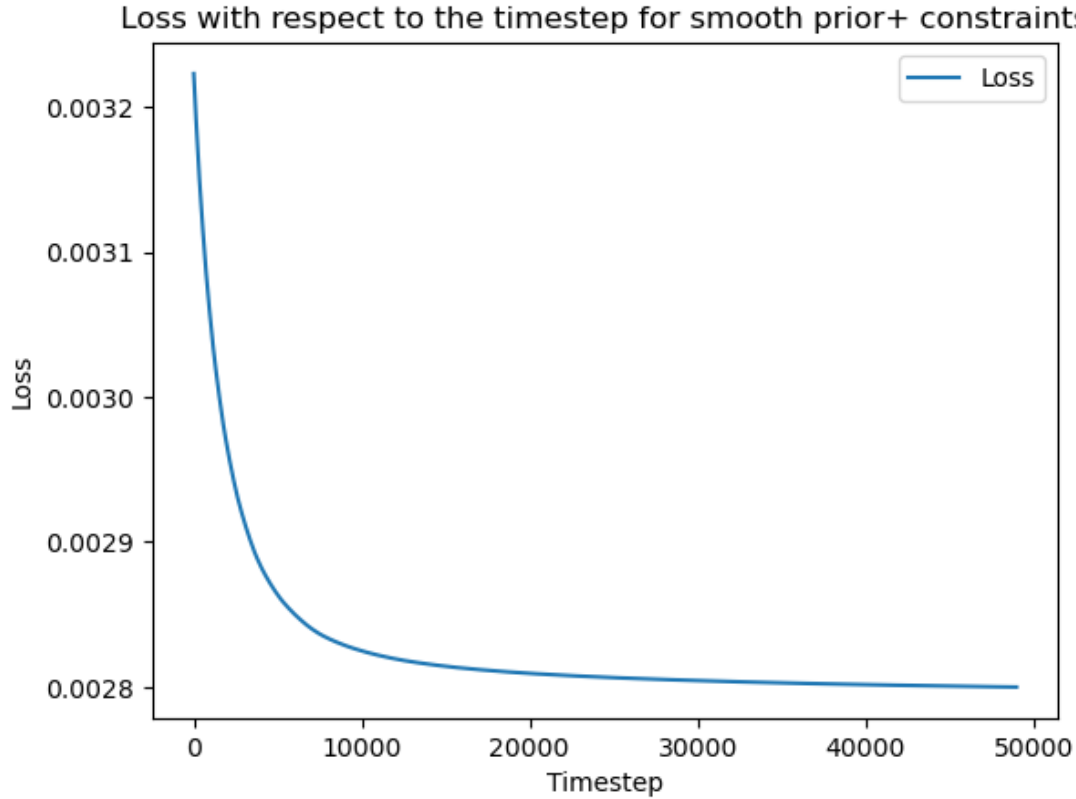
2.2.4 Step 4: Compute normalized quadratic error

```
[21]: errors=normalize_quadratic_error(x_chap=x_chap_cst,x_barre=X)
      print("Normalized quadratic error",errors)
```

Normalized quadratic error 0.18474288387006366

2.2.5 Step 5 : Display loss to check that the algorithm indeed converge

```
[24]: # plot the loss
      title=" Loss with respect to the timestep for smooth prior+ constraints "
      def plot_loss(loss,title):
          plt.plot(loss[1000:],label="Loss")
          plt.xlabel("Timestep")
          plt.ylabel("Loss")
          plt.title(title)
          # set y_log
          #plt.yscale('log')
          plt.legend()
      plot_loss(loss,title)
```



2.3 Sparsity Prior

$$(\forall x \in \mathbb{R}^N) \quad g(x) = \|x\|_1$$

2.3.1 Step 1: Existence and uniqueness of solutions

On note alors

- $f(x) = \frac{1}{2}\|Kx - y\|^2$ est propre, convexe et continue.
- g est propre continue et convexe. Elle est aussi coercive

Donc $f + g$ est continue convexe et f est positive et g coercive, $f+g$ est coercive. Donc $f + g$ admet au moins un minimum.

On a pas nécessairement unicité du minimum car la fonction n'est pas nécessairement strictement convexe (on est dans le cas de la régression LASSO)

2.3.2 Step 2: Propose approaches for solving Problem (4)

On a que

- f est différentiable. Son gradient est ν lipschitzien avec ν la plus grande valeur propre de $(K^T K + D^T D)$

- g est continue, convexe et propre. Elle admet une expression proximable simple et connue d'après le cours

Comme le minimum est non vide on a toutes les hypothèses pour appliquer les algorithmes de *Forward – Backward*

2.3.3 Step 3: Implement solution and display restored signal

```
[25]: class f1_sparsityprior(fun_minimize):
    def __init__(self,K,y):
        self.K=K
        self.y=y
        return
    def grad(self,x:"np.array")->"np.array":
        return self.K.T @ (self.K @ x -self.y )
    def __call__(self,x):
        return 1/2*np.linalg.norm(self.K @ x -self.y)**2

x_test=np.zeros(N)
f=f1_sparsityprior(K,y)
print(f.grad(x_test).shape)
```

(200,)

```
[26]: # Compute the proximal operator of the L1 norm
class g1_sparsityprior(fun_minimize):
    # represent the L1 norm
    def __init__(self,gamma,beta:float=1):

        self.gamma=gamma
        self.beta=beta
        return
    def proximal(self,x:"np.array")->"np.array":
        # compute the prox of xi -> gamma*|xi| ( see slide 28 of ECP_prox)
        # compute the right part of the proximal operator
        #! Beta is missing
        thresh=self.beta*self.gamma
        l1=np.max(
            [np.abs(x)-thresh,
             np.zeros(x.shape)],axis=0
        )
        # compute the sign of the vector
        l2=np.sign(x)
        new_x=np.zeros(len(x))
        for i in range(len(x)):
            if x[i]>thresh:
                new_x[i]=x[i]-thresh
            elif x[i]< -thresh:
```

```

        new_x[i]=x[i]+thresh
    else:
        new_x[i]=0

    return new_x

    return l1*l2
def __call__(self,x):
    return np.linalg.norm(x,1)

# ranom vectors
x_test=np.random.rand(N)
gamma_test=0.1
beta_test=1
g1_test=g1_sparsityprior(gamma_test,beta=beta_test)
# print(g1_test.proximal(x_test).shape)
# print(g1_test(x_test))

```

```

[27]: def forward_backward(x0,gamma,lambda_n,y,num_iter,beta:float):
        x_n=x0
        y_n=x0
        f=g1_sparsityprior(gamma=gamma,beta=beta)
        g=f1_sparsityprior(K,y)
        loss=[]
        for n in range(num_iter):
            y_n=x_n-gamma*g.grad(x=x_n)
            x_n=x_n+lambda_n[n]*(f.proximal(x=y_n)-x_n)
            if n%10000==0:
                print("iteration",n,"the loss is",f(x_n)+g(x_n))
            loss.append(f(x_n)+g(x_n))

        return x_n,loss

```

```

[28]: # init x at random
x0= np.random.rand(N)
x0=np.zeros(N)
# add hyperparameters for the backward forward method
num_iter=50000
beta=0.001
# gamma must be between 0 and 2/v_max
gamma=1/v_max
# lambda must be between 0 nad delta= 2-(gamma*v_max/2)
delta=2-(gamma*v_max/2)
lambda_n=[1/2*delta for k in range(num_iter)]

```

```

[29]: x_chap_sparsity,loss=forward_backward(x0=x0,gamma=gamma,lambda_n=lambda_n,y=y,num_iter=num_ite

```

iteration 0 the loss is 0.7298362618617273

```

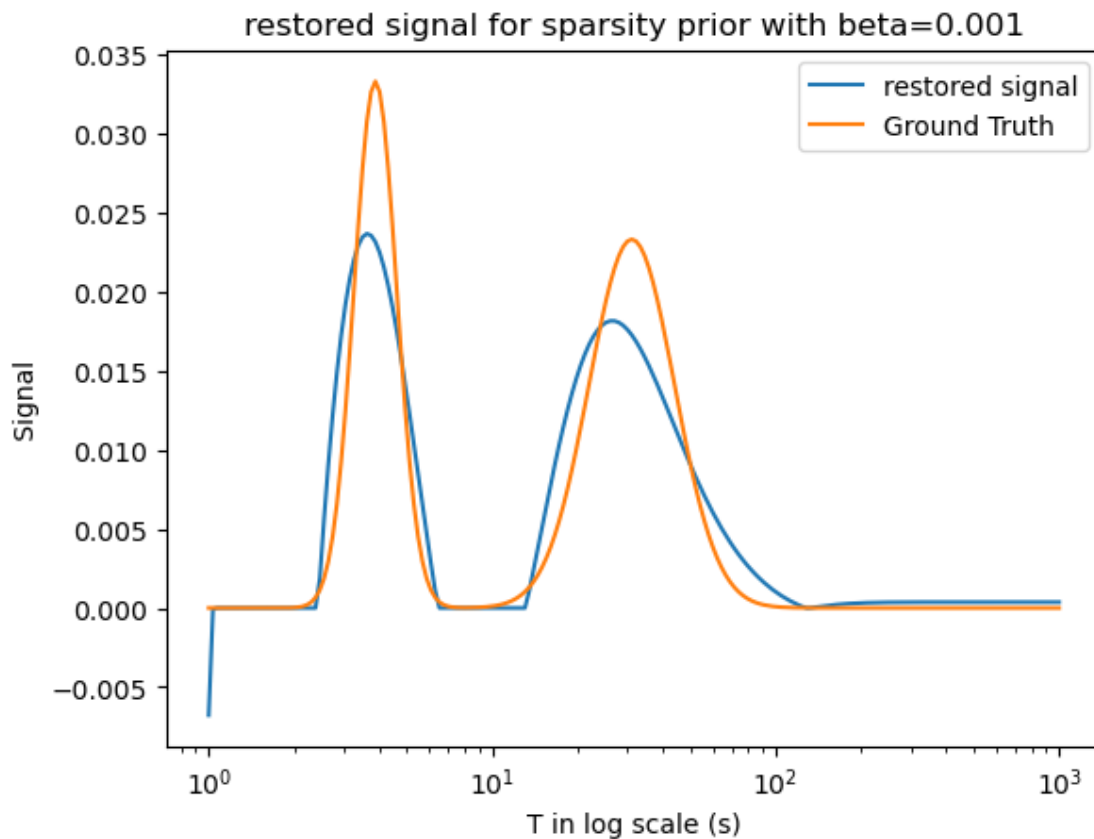
iteration 10000 the loss is 1.0555220262953116
iteration 20000 the loss is 1.0416790781790672
iteration 30000 the loss is 1.0366500062393365
iteration 40000 the loss is 1.033978411562172

```

```

[30]: # plot the signal
title="restored signal for sparsity prior with beta={}".format(beta)
label="restored signal"
plot_signal(T,x_chap_sparsity,title,label=label)
plot_signal(T,X,title,label="Ground Truth")
plt.xscale('log')
#! Seems weird

```



2.3.4 Step 4: Compute normalized quadratic error

```

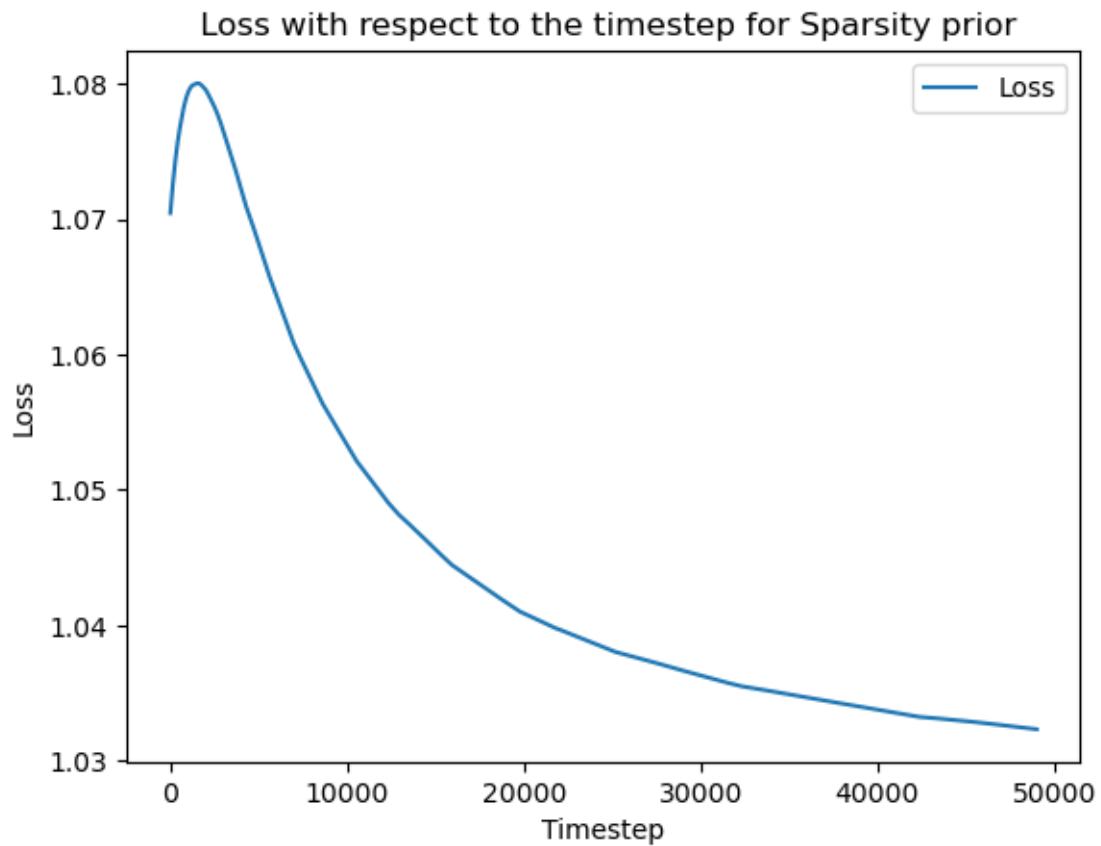
[31]: errors=normalize_quadratic_error(x_chap=x_chap_sparsity,x_barre=X)
print("Normalized quadratic error",errors)

```

Normalized quadratic error 0.29273470648481115

Step 5 : Display loss to check that the algorithm indeed converge

```
[32]: # plot the loss
title=" Loss with respect to the timestep for Sparsity prior"
def plot_loss(loss,title):
    plt.plot(loss[1000:],label="Loss")
    plt.xlabel("Timestep")
    plt.ylabel("Loss")
    plt.title(title)
    # set y_log
    #plt.yscale('log')
    plt.legend()
plot_loss(loss,title)
```



———— FIN PARTIE 1 ————

La suite de la Partie 2 est dans le dossier TP1_part2